PYTHON

From Basics to Advanced

A Topic-by-Topic Learning Handbook

120 topics • 4 learning tracks • explanations, runnable examples, and exercises

LEARN → RUN → EXPLAIN → PRACTICE

Designed for learners who want to understand Python, not merely memorize syntax. Work through the topics in order, run every example, and complete the exercises in your own files.

How to Use This Handbook

This handbook expands the 120 concepts in the supplied reference image. The order and topic names follow that image, grouped into the same four learning tracks. Explanations and examples are added to help you understand and practise each concept.

1. Theory - understand the idea

Read the explanation first. Focus on what the feature does, when it is useful, and what problem it solves.

2. Coding example - run it yourself

Type the example into a small Python file and execute it. Predict the output before you run it, then change values and run it again.

3. Explanation - connect code to behavior

Read the walkthrough to understand the key lines and the result. The goal is to explain the code in your own words.

4. Practice exercise - retrieve the idea

Complete the exercise without copying the example. Add edge cases and test what happens with unusual or empty inputs.

5. Pitfall - learn the common failure mode

Pay special attention to the mistake note. Many problems in real code arise from small misunderstandings such as mutability, input conversion, or file modes.

Recommended Study Routine

A practical session can be 30-60 minutes: read one or two topics, run and modify the examples, solve the exercises, then write a short note explaining the concept. There are 120 topics, so a “30 days” target is possible only if you cover several topics on many days; do not rush at the cost of understanding.

Important Notes

Examples are educational and use standard Python unless a version-specific note is stated. Terminal commands are labelled as terminal commands; do not paste them into a `.py` file. For file examples, run them from a safe practice folder. Some advanced topics (async, threading, multiprocessing, decorators) need additional practice after the first reading.

Contents

Each learning track contains 30 topics. The topic list below follows the sequence in the reference image.

Part 1: Python Basics & Control Flow

01. print()
02. input()
03. Variables
04. int()
05. float()
06. str()
07. bool()
08. type()
09. isinstance()
10. Arithmetic Operators
11. Comparison Operators
12. Logical Operators
13. Assignment Operators
14. Membership Operators
15. Identity Operators
16. if
17. elif
18. else
19. for
20. while
21. range()
22. break
23. continue
24. pass
25. enumerate()
26. zip()
27. len()
28. min()
29. max()
30. sum()

Part 2: Data Structures & Functions

01. Lists
02. Tuples
03. Sets
04. Dictionaries
05. List Slicing
06. append()
07. extend()
08. insert()
09. remove()
10. pop()
11. sort()
12. sorted()
13. reverse()
14. index()
15. count()
16. get()
17. keys()
18. values()
19. items()
20. Set Operations
21. List Comprehension
22. Dict Comprehension
23. Set Comprehension
24. def
25. return
26. Function Arguments
27. Default Arguments
28. *args
29. **kwargs
30. lambda

Part 3: OOP, Files & Errors

01. Classes
02. Objects
03. __init__()
04. self
05. Instance Attributes
06. Class Attributes
07. Instance Methods
08. @classmethod
09. @staticmethod
10. Inheritance
11. Multiple Inheritance
12. Encapsulation
13. Polymorphism
14. Abstraction
15. Method Overriding
16. super()
17. Magic Methods
18. @property
19. open()
20. read()
21. write()
22. with
23. File Modes
24. try
25. except
26. else
27. finally
28. raise
29. Custom Exceptions
30. assert

Part 4: Advanced Python & Ecosystem

01. import
02. from ... import
03. Modules
04. Packages
05. pip
06. Virtual Environments
07. __name__
08. __iter__()
09. iter()
10. next()
11. Generators
12. yield
13. Decorators
14. Closures
15. Recursion
16. Context Managers
17. map()
18. filter()
19. any()
20. all()
21. async
22. await
23. Coroutines
24. asyncio
25. Threading
26. Multiprocessing
27. dataclass
28. Type Hints
29. Regular Expressions
30. JSON Handling

Part 1: Python Basics & Control Flow

Read values, work with data, make decisions, and repeat work with loops.

001. print()

THEORY - understand the idea

Use `print()` to display information in the terminal. It is useful for showing results, checking what a program is doing, and giving feedback to a user. By default, several values passed to `print()` are separated by a space, and the call ends with a newline.

CODING EXAMPLE

```
name = "Latif"
score = 92
print("Student:", name)
print("Score:", score)
print("Keep practicing", end="!")
```

HOW THE CODE WORKS

`print()` converts each value to readable text for display. `sep` changes the separator between multiple values; `end` changes what is written after them. Here the final output ends with `!` rather than a newline.

PRACTICE EXERCISE

Print a product name, price, and stock count on one line separated by `|`. Then print a second line without adding a newline at the end of the first print call.

COMMON PITFALL

`print()` is for display, not for returning a value from a function. A function that prints a result does not automatically give that result back to its caller.

---

002. input()

THEORY - understand the idea

Use `input()` when a program needs text typed by the user. Python displays an optional prompt, waits for input, and returns what the user typed as a string. Convert the value before doing numeric calculations.

CODING EXAMPLE

```
name = input("What is your name? ")
age_text = input("How old are you? ")
age = int(age_text)
print(f"Hello, {name}. Next year you will be {age + 1}.")
```

HOW THE CODE WORKS

Both prompts return strings. `int(age_text)` converts the age text into an integer, and the f-string inserts the values into a readable sentence.

PRACTICE EXERCISE

Ask for a user's city and favorite number. Print a sentence with both values and print the favorite number multiplied by two.

COMMON PITFALL

Input can be empty or invalid. A user typing `twenty` cannot be converted by `int()`; later, learn to validate input and handle conversion errors.

---

003. Variables

THEORY - understand the idea

A variable is a name that refers to a value. Assignment with `=` binds the name to the value on the right. Python is dynamically typed: a name can later refer to a value of another type, although changing types casually can make code harder to understand.

CODING EXAMPLE

```
product_name = "Notebook"
price = 250.0
quantity = 3
total = price * quantity
print(product_name, total)
```

HOW THE CODE WORKS

Each name labels a value that can be reused. `total` is calculated from `price` and `quantity`, so changing either input updates the result the next time the code runs.

PRACTICE EXERCISE

Create variables for a book title, its price, and the number of copies. Calculate and print the total bill.

COMMON PITFALL

Use descriptive names such as `total_price`, not cryptic names like `tp` unless the meaning is obvious. Names are case-sensitive: `price` and `Price` are different.

---

004. int()

THEORY - understand the idea

The `int()` constructor creates an integer from a compatible value. It can convert strings such as `"42"` and truncate a floating-point number toward zero. It does not round a float to the nearest integer.

CODING EXAMPLE

```
print(int("42"))
print(int(8.9))
print(int(-8.9))
```

HOW THE CODE WORKS

`int("42")` parses digits into the integer `42`. The float conversions become `8` and `-8`; the fractional parts are discarded toward zero.

PRACTICE EXERCISE

Convert the string `"2026"` to an integer, then add five. Try converting `"20.5"` with `int()` and explain the error.

COMMON PITFALL

`int("20.5")` raises `ValueError`; convert to `float` first if decimal text is expected, then decide how to handle the fraction.

---

005. float()

THEORY - understand the idea

The `float()` constructor represents numbers that can have a fractional part. It converts compatible numeric strings and integers to floating-point values. Floating-point arithmetic is efficient but can represent some decimal fractions only approximately.

CODING EXAMPLE

```
price_text = "19.95"
price = float(price_text)
quantity = 3
print(price * quantity)
print(float(7))
```

HOW THE CODE WORKS

The price text becomes a floating-point number, allowing multiplication. `float(7)` becomes `7.0`. Small display differences can occur because binary floating point cannot precisely store every decimal fraction.

PRACTICE EXERCISE

Ask for a distance in kilometers as text, convert it to a float, and calculate the distance in meters.

COMMON PITFALL

For financial systems, do not rely on binary floating-point values for exact currency calculations; learn about `decimal.Decimal` when needed.

---

006. str()

THEORY - understand the idea

The `str()` constructor returns a readable string representation of a value. It is useful when combining data with text or preparing values for display. String conversion does not mean the original value changes type.

CODING EXAMPLE

```
age = 20
message = "Age: " + str(age)
print(message)
print(type(age))
```

HOW THE CODE WORKS

`str(age)` converts the integer into text so it can be concatenated with another string. The original variable `age` remains an integer.

PRACTICE EXERCISE

Convert an integer order number into text and build a message such as `Order #104 is ready`.

COMMON PITFALL

You usually do not need manual `str()` conversion inside an f-string: `f"Age: {age}"` is clearer.

---

007. bool()

THEORY - understand the idea

The `bool()` constructor converts a value to `True` or `False`. Values such as `0`, `0.0`, `""`, `None`, and empty collections are falsey; most other values are truthy. Truthiness is frequently used in conditions.

CODING EXAMPLE

```
print(bool(0))
print(bool(7))
print(bool(""))
print(bool("False"))
print(bool([]))
```

HOW THE CODE WORKS

Zero, an empty string, and an empty list are falsey. The non-empty string `"False"` is truthy because Python checks whether the string is empty, not whether its text spells a false value.

PRACTICE EXERCISE

Predict the truth value of `0`, `-1`, `""`, `"0"`, `[]`, and `[0]` before running each through `bool()`.

COMMON PITFALL

Do not parse user-provided text with `bool(text)` to interpret yes/no. Any non-empty string—including `"no"`—is truthy. Compare normalized text explicitly.

---

008. type()

THEORY - understand the idea

Use `type(value)` to inspect the exact type of a value. This can help during learning and debugging. In most application code, `isinstance()` is more flexible because it can also accept subclasses.

CODING EXAMPLE

```
value = [1, 2, 3]
print(type(value))
print(type(12))
print(type("hello"))
```

HOW THE CODE WORKS

Each call returns a type object such as `list`, `int`, or `str`; `print()` displays its representation. The value itself is not changed.

PRACTICE EXERCISE

Create one value each of type `int`, `float`, `str`, `bool`, `list`, and `dict`. Print each value alongside its type.

COMMON PITFALL

Prefer `isinstance(value, SomeType)` when asking whether a value belongs to a type family. Exact-type comparisons can reject valid subclasses.

---

009. isinstance()

THEORY - understand the idea

Use `isinstance(value, Type)` to check whether a value is an instance of a type or its subclass. It returns a Boolean, so it works naturally in conditions. It can also accept a tuple of types.

CODING EXAMPLE

```
value = 12
if isinstance(value, (int, float)):
    print("This is numeric")

print(isinstance("hello", str))
```

HOW THE CODE WORKS

The first check succeeds because `12` is an integer and `int` is in the tuple. The second check returns `True` because the value is a string.

PRACTICE EXERCISE

Write a function that accepts either an integer or a float and reports `numeric`; for all other types, report `not numeric`. Decide how you want to treat `True`, since `bool` is a subclass of `int`.

COMMON PITFALL

In Python, `isinstance(True, int)` is `True`. Exclude booleans separately if your program needs numbers but not true/false values.

---

010. Arithmetic Operators

THEORY - understand the idea

Arithmetic operators perform mathematical operations: `+`, `-`, `*`, `/`, `//`, `%`, and `**`. Division `/` produces a float; floor division `//` rounds the quotient down toward negative infinity; `%` gives the remainder; `**` raises a value to a power.

CODING EXAMPLE

```
a, b = 17, 5
print(a + b, a - b, a * b)
print(a / b, a // b, a % b)
print(2 ** 3)
```

HOW THE CODE WORKS

The results are `22`, `12`, `85`, `3.4`, `3`, `2`, and `8`. These operators follow standard precedence rules; parentheses can make intended grouping explicit.

PRACTICE EXERCISE

Given `minutes = 137`, calculate whole hours and remaining minutes using `//` and `%`.

COMMON PITFALL

Floor division is not the same as truncating toward zero for negative numbers. For example, `-7 // 3` is `-3`.

---

011. Comparison Operators

THEORY - understand the idea

Comparison operators compare two values and return a Boolean: `==`, `!=`, `<`, `<=`, `>`, and `>=`. Comparisons are commonly used in `if` statements and loops. Chained comparisons such as `0 <= score <= 100` are valid in Python.

CODING EXAMPLE

```
score = 86
print(score >= 50)
print(score == 100)
print(0 <= score <= 100)
```

HOW THE CODE WORKS

The expressions evaluate to `True`, `False`, and `True`. `==` checks value equality; it is different from `=` which assigns a value to a name.

PRACTICE EXERCISE

Ask for a temperature and print whether it is below zero, exactly zero, or above zero using comparisons.

COMMON PITFALL

Do not write `score = 50` when you mean to compare. Use `==`; the assignment operator changes a variable.

---

012. Logical Operators

THEORY - understand the idea

The operators `and`, `or`, and `not` combine or reverse truth values. `and` is true when both sides are truthy; `or` is true when at least one side is truthy; `not` reverses truthiness. Python short-circuits: it may skip evaluating the second operand when the result is already known.

CODING EXAMPLE

```
age = 20
has_ticket = True
print(age >= 18 and has_ticket)
print(age < 18 or not has_ticket)
```

HOW THE CODE WORKS

The first condition is true because both checks are true. The second is false because both `age < 18` and `not has_ticket` are false.

PRACTICE EXERCISE

Create a condition for access that requires a user to be at least 18 and have an active account. Test several combinations.

COMMON PITFALL

Use parentheses when a condition combines `and` and `or` and the grouping is not immediately clear.

---

013. Assignment Operators

THEORY - understand the idea

Assignment operators bind values to names. `=` performs ordinary assignment; augmented forms such as `+=`, `-=`, `*=`, and `/=` update an existing variable using an operation. They make repeated updates concise.

CODING EXAMPLE

```
balance = 100
balance += 25
balance -= 10
balance *= 2
print(balance)
```

HOW THE CODE WORKS

The balance changes from `100` to `125`, then `115`, then `230`. These statements update the variable, rather than creating a separate expression result that is discarded.

PRACTICE EXERCISE

Start with `count = 0`, add 3 five times using `+=`, and print the final value.

COMMON PITFALL

Augmented assignment can mutate mutable objects in place in cases such as lists. Understand whether an operation changes an object or rebinds a name.

---

014. Membership Operators

THEORY - understand the idea

The operators `in` and `not in` check whether a value is present in a container or substring. They work with strings, lists, tuples, sets, and dictionary keys, though performance differs by data structure.

CODING EXAMPLE

```
allowed = {"admin", "editor"}
role = "editor"
print(role in allowed)
print("py" in "python")
print("missing" not in allowed)
```

HOW THE CODE WORKS

The first and second checks are true; the last is also true because `missing` is not one of the set elements. For dictionaries, `in` checks keys by default.

PRACTICE EXERCISE

Create a list of three favorite foods. Ask for a food and report whether it is in the list.

COMMON PITFALL

Checking membership in a list generally scans elements one by one; a set is often a better choice for frequent membership tests.

---

015. Identity Operators

THEORY - understand the idea

The identity operators `is` and `is not` test whether two names refer to the same object. They do not test whether two values are equal. Use `==` for value comparison and use `is` most commonly for identity checks against singletons such as `None`.

CODING EXAMPLE

```
a = [1, 2]
b = [1, 2]
c = a
print(a == b)
print(a is b)
print(a is c)
result = None
print(result is None)
```

HOW THE CODE WORKS

`a` and `b` have equal contents but are distinct list objects, so equality is true and identity is false. `c` references the same list as `a`, and `result is None` is the idiomatic null check.

PRACTICE EXERCISE

Create two equal but separate lists and a third name pointing at the first. Predict `==` and `is` results.

COMMON PITFALL

Do not use `is` to compare strings or numbers by value. Any apparent interning behavior is an implementation detail, not a reliable equality rule.

---

016. if

THEORY - understand the idea

An `if` statement runs an indented block only when its condition is truthy. It allows a program to make a decision. The condition can be a comparison, a Boolean expression, or another value with truthiness.

CODING EXAMPLE

```
stock = 4
if stock > 0:
    print("Available")
    print("You can place an order")
```

HOW THE CODE WORKS

Both indented lines run when `stock > 0` evaluates to true. Indentation defines the block, so consistent four-space indentation is the standard style.

PRACTICE EXERCISE

Ask the user for a number. If the number is positive, print `positive`; initially write only this branch.

COMMON PITFALL

A colon is required after the condition, and the following block must be indented.

---

017. elif

THEORY - understand the idea

`elif` means "else if." It checks another condition only when earlier `if`/`elif` conditions were false. Python runs the first matching branch and skips the remaining branches, making branch order important.

CODING EXAMPLE

```
score = 74
if score >= 90:
    grade = "A"
elif score >= 70:
    grade = "B"
else:
    grade = "C or below"
print(grade)
```

HOW THE CODE WORKS

A score of 74 fails the first test and passes the second, so `grade` becomes `B`. The conditions should be ordered to match the intended priority.

PRACTICE EXERCISE

Extend a grade calculator to use separate branches for A, B, C, D, and F. Test values exactly at each boundary.

COMMON PITFALL

If a broad condition comes first, it can make later branches unreachable. For grades, check the highest threshold first.

---

018. else

THEORY - understand the idea

In an `if` chain, `else` is the fallback branch when no earlier condition matched. In loops, a loop-level `else` has a different meaning: it runs if the loop finishes normally without a `break`. Context matters.

CODING EXAMPLE

```
temperature = 12
if temperature > 25:
    print("Hot")
else:
    print("Not hot")

for n in range(3):
    print(n)
else:
    print("Loop completed")
```

HOW THE CODE WORKS

The conditional `else` handles the false condition. The loop `else` runs because the `for` loop reached the end normally and was not stopped by `break`.

PRACTICE EXERCISE

Write a search loop over a list. Print `Found` and break when the target appears; use loop `else` to print `Not found` only when the search completes.

COMMON PITFALL

Loop `else` is easy to confuse with an `if` fallback. Use it only when the normal-completion behavior makes the code clearer.

---

019. for

THEORY - understand the idea

A `for` loop visits items from an iterable, one at a time. It is suitable for lists, strings, dictionaries, ranges, files, and many other objects. Prefer it when you know the collection you want to process.

CODING EXAMPLE

```
names = ["Amina", "Bilal", "Sara"]
for name in names:
    print(f"Hello, {name}!")
```

HOW THE CODE WORKS

Each iteration assigns the next list element to `name`, then runs the body. The loop ends after every item has been processed.

PRACTICE EXERCISE

Loop through a list of prices and calculate the total without using the built-in `sum()` yet.

COMMON PITFALL

You usually should not change the list you are iterating over unless you have a clear plan; remove or append operations can cause skipped or unexpected iterations.

---

020. while

THEORY - understand the idea

A `while` loop repeats as long as its condition remains truthy. It is useful when the number of repetitions depends on a changing state or user input. Something in the loop usually needs to move the condition toward becoming false.

CODING EXAMPLE

```
remaining = 3
while remaining > 0:
    print("Attempts left:", remaining)
    remaining -= 1
print("Finished")
```

HOW THE CODE WORKS

The condition is checked before each iteration. `remaining -= 1` reduces the value until the condition is false, after which the loop ends.

PRACTICE EXERCISE

Create a number-guessing loop with a secret number. Keep asking until the user guesses correctly; add a way to quit.

COMMON PITFALL

If the condition never becomes false and there is no `break`, the loop can run forever.

---

021. range()

THEORY - understand the idea

`range()` produces an immutable sequence of integers, commonly used to control loops. Its stop value is excluded. `range(stop)`, `range(start, stop)`, and `range(start, stop, step)` cover common patterns.

CODING EXAMPLE

```
for n in range(1, 6):
    print(n)

for even in range(2, 11, 2):
    print(even)
```

HOW THE CODE WORKS

The first loop prints 1 through 5 because 6 is excluded. The second starts at 2 and increases by 2, printing even values up to 10.

PRACTICE EXERCISE

Print the numbers from 10 down to 1 using a negative step. Then print all multiples of 3 below 30.

COMMON PITFALL

If the step is zero, `range()` raises `ValueError`. For descending sequences, use a negative step and a stop value that fits the exclusive-stop rule.

---

022. break

THEORY - understand the idea

`break` exits the nearest enclosing loop immediately. It is useful when a search has found its target or a stopping condition occurs before the loop naturally finishes.

CODING EXAMPLE

```
numbers = [4, 8, 15, 16, 23, 42]
for number in numbers:
    if number > 20:
        print("First value above 20:", number)
        break
```

HOW THE CODE WORKS

The loop prints `23` and then exits; `42` is never examined. Only the innermost loop is exited if loops are nested.

PRACTICE EXERCISE

Search a list for a target value. Stop at the first match and report how many elements were checked.

COMMON PITFALL

Use `break` for a real stopping condition rather than adding many nested flags that make loop flow hard to follow.

---

023. continue

THEORY - understand the idea

`continue` skips the rest of the current iteration and moves to the next item or condition check. It is useful for ignoring values that do not need processing while keeping the loop structure simple.

CODING EXAMPLE

```
for number in range(1, 8):
    if number % 2 == 0:
        continue
    print(number)
```

HOW THE CODE WORKS

Even numbers trigger `continue`, so the `print()` line is skipped for them. Only odd numbers are printed.

PRACTICE EXERCISE

Loop through a list of numbers and print only positive values, using `continue` to skip zero and negatives.

COMMON PITFALL

In a `while` loop, ensure the state update happens before a `continue` branch or you may accidentally create an infinite loop.

---

024. pass

THEORY - understand the idea

`pass` is a no-operation statement. Python requires an indented suite after constructs such as `if`, `def`, and `class`; `pass` lets you leave that block empty temporarily while keeping the program syntactically valid.

CODING EXAMPLE

```
def future_feature():
    pass

score = 55
if score >= 50:
    pass  # Add the success behavior later
else:
    print("Try again")
```

HOW THE CODE WORKS

The function exists but does nothing when called. The `if` success branch is also intentionally empty; the `else` branch prints for a failing score.

PRACTICE EXERCISE

Create a function named `calculate_discount()` with a valid body later in mind. For now, use `pass` and then replace it with an implementation.

COMMON PITFALL

`pass` does not skip a loop iteration; `continue` does. It also does not mean “to be implemented” to Python—it really does nothing.

---

025. enumerate()

THEORY - understand the idea

`enumerate()` produces pairs containing an index and an item from an iterable. It is often clearer than manually incrementing a counter, and `start=` can make the displayed numbering begin at 1 or another value.

CODING EXAMPLE

```
tasks = ["study", "exercise", "review"]
for number, task in enumerate(tasks, start=1):
    print(number, task)
```

HOW THE CODE WORKS

Each iteration unpacks the pair into `number` and `task`. The index is generated by `enumerate`; the list does not need to store index values.

PRACTICE EXERCISE

Print a numbered menu from a list of at least four choices, starting at 1.

COMMON PITFALL

Do not confuse the index with a value stored in the list. If you only need values, a plain `for item in items` loop is simpler.

---

026. zip()

THEORY - understand the idea

`zip()` combines iterables in parallel, yielding tuples with one item from each. It stops when the shortest iterable is exhausted by default. This is convenient when related data is stored in separate sequences.

CODING EXAMPLE

```
names = ["Amina", "Bilal", "Sara"]
scores = [88, 76, 91]
for name, score in zip(names, scores):
    print(name, score)
```

HOW THE CODE WORKS

Each pass pairs a name with the score at the same position. If the lists have different lengths, ordinary `zip()` silently stops at the shorter one.

PRACTICE EXERCISE

Keep two lists of product names and prices. Use `zip()` to print each product with its price; then test lists of different lengths.

COMMON PITFALL

When mismatched lengths should be considered a bug, use `zip(..., strict=True)` in Python 3.10+ to catch the mismatch.

---

027. len()

THEORY - understand the idea

`len()` returns the number of items in a container, such as a string, list, tuple, set, or dictionary. For a string, the result counts Unicode code points, not necessarily user-perceived characters or bytes.

CODING EXAMPLE

```
message = "hello"
items = [10, 20, 30]
profile = {"name": "Latif", "active": True}
print(len(message))
print(len(items))
print(len(profile))
```

HOW THE CODE WORKS

The outputs are 5, 3, and 2. A dictionary length counts keys, and nested elements do not get counted recursively.

PRACTICE EXERCISE

Ask for a sentence and print the number of characters. Then create a list of items and print how many items it contains.

COMMON PITFALL

`len()` is not a measure of the size of a nested structure or the amount of memory it uses; it reports the top-level item count.

---

028. min()

THEORY - understand the idea

`min()` returns the smallest item in an iterable or the smallest of multiple arguments. It can compare numbers and other mutually orderable values. Use the `key=` argument when the comparison should be based on a property.

CODING EXAMPLE

```
temps = [18, 24, 11, 20]
print(min(temps))
words = ["pear", "fig", "watermelon"]
print(min(words, key=len))
```

HOW THE CODE WORKS

The smallest temperature is `11`. With `key=len`, Python compares the lengths of the words and returns `fig`, the shortest one.

PRACTICE EXERCISE

Find the cheapest price in a list. Then find the shortest word in a list of words.

COMMON PITFALL

An empty iterable raises `ValueError` unless you provide a `default` with the iterable form of `min()`.

---

029. max()

THEORY - understand the idea

`max()` is the counterpart of `min()`: it returns the greatest item or greatest of several arguments. Like `min()`, it supports `key=` and can accept a `default` when the iterable is empty.

CODING EXAMPLE

```
scores = [72, 95, 88, 64]
print(max(scores))
products = [{"name": "Pen", "price": 20},
            {"name": "Bag", "price": 900}]
print(max(products, key=lambda item: item["price"]))
```

HOW THE CODE WORKS

`max(scores)` returns 95. The `key` function tells Python to compare each dictionary by its `price` value; the full dictionary with the highest price is returned.

PRACTICE EXERCISE

Find the oldest age in a list. Then choose the most expensive product from a list of product dictionaries.

COMMON PITFALL

`max()` returns the item, not necessarily its key value. Access the desired field after selecting the item.

---

030. sum()

THEORY - understand the idea

`sum()` adds the numeric items in an iterable, starting from zero unless a different `start` value is supplied. It is concise and usually clearer than writing a manual accumulation loop for a straightforward total.

CODING EXAMPLE

```
prices = [120, 80, 45]
subtotal = sum(prices)
print(subtotal)
print(sum(prices, start=10))
```

HOW THE CODE WORKS

The first call returns 245. The second begins at 10 and returns 255. `sum()` is intended for numeric addition, not joining strings.

PRACTICE EXERCISE

Calculate a shopping-cart subtotal from a list of prices. Then calculate the average by dividing the sum by the number of items, taking care of an empty list.

COMMON PITFALL

Dividing by `len(items)` fails when the list is empty. Decide what your program should do when there are no values.

Part 2: Data Structures & Functions

Store groups of data, transform collections, and organize reusable logic.

031. Lists

THEORY - understand the idea

A list is an ordered, mutable collection. It can contain duplicate values and values of different types, though lists of one consistent type are often easier to work with. Lists support indexing, slicing, iteration, and many useful methods.

CODING EXAMPLE

```
tasks = ["read", "code", "test"]
tasks[1] = "debug"
tasks.append("review")
print(tasks)
print(tasks[0])
```

HOW THE CODE WORKS

Indexing begins at zero, so `tasks[1]` refers to the second element. Assignment changes that item; `append()` adds a new item at the end.

PRACTICE EXERCISE

Create a list of five goals. Print the first and last goals, replace one item, and add a sixth goal.

COMMON PITFALL

Lists are mutable. If two variables refer to the same list, changing it through either name changes the same underlying object.

---

032. Tuples

THEORY - understand the idea

A tuple is an ordered collection that cannot have its elements reassigned after creation. Tuples are useful for fixed groups of related values and can be unpacked into variables. A tuple is only deeply immutable if its elements are also immutable.

CODING EXAMPLE

```
point = (4, 7)
x, y = point
print(x, y)
print(point[0])
# A one-item tuple needs a trailing comma:
single = (5,)
```

HOW THE CODE WORKS

The coordinates unpack into `x` and `y`. `point[0]` reads the first value. Parentheses alone do not make a one-item tuple; the comma does.

PRACTICE EXERCISE

Represent a date as a tuple `(year, month, day)`. Unpack it and print a formatted date.

COMMON PITFALL

You cannot assign `point[0] = 10`. If you need to update individual elements, consider a list or a suitable data class.

---

033. Sets

THEORY - understand the idea

A set stores unique, hashable elements and does not preserve a meaningful positional index. Sets are useful for removing duplicates and fast membership checks. Empty sets are created with `set()`, because `{}` creates an empty dictionary.

CODING EXAMPLE

```
languages = {"Python", "Go", "Python", "Rust"}
print(languages)
print("Go" in languages)
languages.add("Java")
print(len(languages))
```

HOW THE CODE WORKS

Duplicates collapse to one element. Membership checks whether `Go` exists, and `add()` inserts a new unique element. Output order should not be relied on as a positional order.

PRACTICE EXERCISE

Given a list of repeated email addresses, create a set to find the unique addresses. Print how many unique addresses remain.

COMMON PITFALL

Set elements must be hashable. A list cannot be an element of a set, but a tuple of hashable values can.

---

034. Dictionaries

THEORY - understand the idea

A dictionary maps unique, hashable keys to values. It is useful when data has named fields, such as a user profile or product. Dictionaries preserve insertion order in modern Python, but keys are used for lookup rather than numeric positions.

CODING EXAMPLE

```
product = {"name": "Keyboard", "price": 3500, "stock": 12}
print(product["name"])
product["stock"] -= 1
product["category"] = "Accessories"
print(product)
```

HOW THE CODE WORKS

Square brackets retrieve a value for a key and can also assign or update it. A missing key raises `KeyError`; the `get()` method is useful when a key may be absent.

PRACTICE EXERCISE

Create a dictionary for a book with title, author, and year. Update the year and add a `pages` key.

COMMON PITFALL

Dictionary keys must be hashable and unique. Assigning to an existing key replaces its old value.

---

035. List Slicing

THEORY - understand the idea

Slicing extracts a portion of a sequence using `sequence[start:stop:step]`. The start index is included and the stop index is excluded. Omitting values uses defaults; negative indexes count from the end.

CODING EXAMPLE

```
numbers = [10, 20, 30, 40, 50, 60]
print(numbers[1:4])
print(numbers[:3])
print(numbers[-2:])
print(numbers[::-1])
```

HOW THE CODE WORKS

The slices are `[20, 30, 40]`, the first three items, the final two items, and a reversed copy respectively. Ordinary slicing creates a new list rather than a view.

PRACTICE EXERCISE

From a list of ten values, extract items 3 through 6 (human counting), the final three values, and every second value.

COMMON PITFALL

Python indexes from zero and excludes the stop. Slicing outside the list bounds is usually safe and simply returns available items.

---

036. append()

THEORY - understand the idea

`list.append(item)` adds one object to the end of a list. The item can itself be another list or any other object. The list changes in place, and the method returns `None`.

CODING EXAMPLE

```
numbers = [1, 2]
result = numbers.append(3)
print(numbers)
print(result)
```

HOW THE CODE WORKS

`numbers` becomes `[1, 2, 3]`. `result` is `None`, which is why you should not write `numbers = numbers.append(3)`.

PRACTICE EXERCISE

Start with an empty list. Append three user-entered favorite movies, then print the final list.

COMMON PITFALL

`append([4, 5])` adds one nested list element; it does not add the values `4` and `5` individually.

---

037. extend()

THEORY - understand the idea

`list.extend(iterable)` adds each item from an iterable to the end of a list. It is appropriate when combining the individual contents of another list or other iterable into an existing list.

CODING EXAMPLE

```
numbers = [1, 2]
numbers.extend([3, 4])
numbers.extend((5, 6))
print(numbers)
```

HOW THE CODE WORKS

The resulting list is `[1, 2, 3, 4, 5, 6]`. Each element of the supplied iterable is added separately.

PRACTICE EXERCISE

Combine two lists of weekdays into one list using `extend()`. Compare the result with using `append()` on the second list.

COMMON PITFALL

Extending with a string adds its individual characters. Use that behavior intentionally, or pass a list containing the whole string instead.

---

038. insert()

THEORY - understand the idea

`list.insert(index, item)` inserts an item before the specified index and shifts later elements to the right. It is useful when an item must be placed at a known position instead of always added at the end.

CODING EXAMPLE

```
tasks = ["code", "test"]
tasks.insert(1, "design")
print(tasks)
```

HOW THE CODE WORKS

The new list is `["code", "design", "test"]`; index 1 is the position before the old second element. An index beyond the ends is clamped to the start or end.

PRACTICE EXERCISE

Insert `"urgent"` at the start of a task list and insert a new task between two existing tasks.

COMMON PITFALL

Repeated insertion at the beginning can be inefficient for large lists because existing elements must shift. Consider `collections.deque` for queue-like workloads.

---

039. remove()

THEORY - understand the idea

`list.remove(value)` removes the first element equal to a specified value. It changes the original list and returns `None`. If the value does not exist, it raises `ValueError`.

CODING EXAMPLE

```
colors = ["red", "blue", "red", "green"]
colors.remove("red")
print(colors)
```

HOW THE CODE WORKS

Only the first `red` is removed; the second remains. The method removes by value, not by index.

PRACTICE EXERCISE

Remove a chosen task from a list. Handle the case where the task name is not present without crashing.

COMMON PITFALL

Check membership before removal or use `try`/`except ValueError` when missing values are expected.

---

040. pop()

THEORY - understand the idea

`list.pop(index)` removes and returns the item at an index. With no index, it removes and returns the last item. It is useful when you need both to remove a value and use that value afterwards.

CODING EXAMPLE

```
tasks = ["design", "code", "test"]
last_task = tasks.pop()
first_task = tasks.pop(0)
print(last_task, first_task)
print(tasks)
```

HOW THE CODE WORKS

The last task is removed first, then the item at index zero. The remaining list contains only `code`.

PRACTICE EXERCISE

Build a small stack of three items. Pop and print the latest item, then show the remaining stack.

COMMON PITFALL

Popping from an empty list raises `IndexError`. Check the list or define what an empty stack should mean.

---

041. sort()

THEORY - understand the idea

`list.sort()` sorts the existing list in place and returns `None`. It supports ascending order by default, `reverse=True`, and a `key` function to control what is compared.

CODING EXAMPLE

```
names = ["Sara", "Ali", "Fatima"]
names.sort()
print(names)
names.sort(key=len, reverse=True)
print(names)
```

HOW THE CODE WORKS

The first sort alphabetizes the names. The second sorts by length, longest first. The list itself is changed by each call.

PRACTICE EXERCISE

Sort a list of integers ascending and descending. Then sort a list of words from shortest to longest.

COMMON PITFALL

Do not assign the result of `sort()` to the same list: `sort()` returns `None`. Use `sorted()` when you need a separate new list.

---

042. sorted()

THEORY - understand the idea

`sorted(iterable)` returns a new list in sorted order without modifying the original iterable. It accepts the same useful `key` and `reverse` parameters as `list.sort()` and can sort tuples, strings, sets, and more.

CODING EXAMPLE

```
scores = [70, 95, 82]
ranked = sorted(scores, reverse=True)
print(ranked)
print(scores)
```

HOW THE CODE WORKS

`ranked` is a new descending list, while `scores` remains in its original order. This makes `sorted()` useful when the source order must be preserved.

PRACTICE EXERCISE

Sort a tuple of numbers without converting the original variable into a list. Sort words by length and keep the original list unchanged.

COMMON PITFALL

The result of `sorted()` is a list, regardless of the input iterable type.

---

043. reverse()

THEORY - understand the idea

`list.reverse()` reverses the order of elements in the existing list. It is not the same as sorting descending; it simply flips the current sequence order. It returns `None`.

CODING EXAMPLE

```
steps = ["first", "second", "third"]
steps.reverse()
print(steps)
```

HOW THE CODE WORKS

The list becomes `["third", "second", "first"]`. No alphabetic or numeric comparison takes place.

PRACTICE EXERCISE

Reverse a list of dates and compare the result with `sorted(dates, reverse=True)`. Explain why they can differ.

COMMON PITFALL

Use `reverse()` when you want the opposite of the current order. Use a sort when you want ordering based on values.

---

044. index()

THEORY - understand the idea

`list.index(value)` returns the index of the first matching value. It raises `ValueError` when no match exists. Optional `start` and `stop` arguments limit the search range.

CODING EXAMPLE

```
colors = ["red", "blue", "green", "blue"]
print(colors.index("blue"))
print(colors.index("blue", 2))
```

HOW THE CODE WORKS

The first call returns index 1. The second begins searching at index 2 and returns index 3, the next match.

PRACTICE EXERCISE

Find the position of a chosen name in a list. Print a helpful message if the name is absent.

COMMON PITFALL

`index()` finds the first match only. If values may repeat and you need every position, loop with `enumerate()`.

---

045. count()

THEORY - understand the idea

`list.count(value)` returns how many times a value appears in the list. It is useful for simple frequency checks; for many different values, `collections.Counter` is often more efficient and expressive.

CODING EXAMPLE

```
votes = ["yes", "no", "yes", "yes", "abstain"]
print(votes.count("yes"))
print(votes.count("no"))
```

HOW THE CODE WORKS

The method scans the list and counts values equal to the argument. It does not change the list.

PRACTICE EXERCISE

Count how many times a target number occurs in a list. Extend the program to report whether the number never appears.

COMMON PITFALL

Calling `count()` repeatedly for many different values rescans the list. Use `Counter` for a full frequency table.

---

046. get()

THEORY - understand the idea

`dict.get(key, default)` retrieves a dictionary value without raising `KeyError` when the key is absent. If the key does not exist, it returns the optional default, or `None` if no default is given.

CODING EXAMPLE

```
user = {"name": "Latif", "role": "student"}
print(user.get("name"))
print(user.get("email"))
print(user.get("city", "Not provided"))
```

HOW THE CODE WORKS

The existing `name` is returned, the absent `email` returns `None`, and the absent `city` returns the fallback string. The dictionary is not modified.

PRACTICE EXERCISE

Given a product dictionary, print its stock value or `0` if the `stock` key does not exist.

COMMON PITFALL

`get()` is for safe lookup; it does not create a missing key or store the default in the dictionary.

---

047. keys()

THEORY - understand the idea

`dict.keys()` returns a dynamic view of the dictionary keys. The view reflects later changes to the dictionary and can be iterated over. You can also use `key in dictionary` directly for membership testing.

CODING EXAMPLE

```
student = {"name": "Sara", "year": 2, "active": True}
for key in student.keys():
    print(key)
print("year" in student)
```

HOW THE CODE WORKS

The loop displays each key in insertion order. Membership testing on the dictionary itself is the usual concise choice when checking for a key.

PRACTICE EXERCISE

Print all keys in a product dictionary, one per line. Then check whether `price` exists.

COMMON PITFALL

Do not modify dictionary keys while iterating through a view if doing so changes its size; iterate over `list(dictionary)` when a snapshot is needed.

---

048. values()

THEORY - understand the idea

`dict.values()` returns a dynamic view of a dictionary's values. Use it when you want to process data without caring about key names. The view is iterable but is not a list by itself.

CODING EXAMPLE

```
prices = {"pen": 20, "book": 150, "bag": 900}
for price in prices.values():
    print(price)
print(sum(prices.values()))
```

HOW THE CODE WORKS

The loop visits each stored price. `sum()` can consume the values view directly without making a separate list.

PRACTICE EXERCISE

Create a dictionary of monthly expenses and calculate the total using `values()` and `sum()`.

COMMON PITFALL

Values can repeat and need not be unique. If you need to know which value belongs to which key, use `items()`.

---

049. items()

THEORY - understand the idea

`dict.items()` returns a view of key-value pairs. It is especially useful for loops because each pair can be unpacked into a key and a value. It avoids separately looking up every key.

CODING EXAMPLE

```
inventory = {"pen": 12, "book": 7, "bag": 3}
for product, stock in inventory.items():
    print(f"{product}: {stock} left")
```

HOW THE CODE WORKS

Each item pair is unpacked into `product` and `stock`. The loop connects each product name to its matching quantity.

PRACTICE EXERCISE

Print each course name and student count from a dictionary. Also find the course with the largest count.

COMMON PITFALL

Do not expect `items()` to be a list; it is a dynamic view. Convert it to `list()` if you specifically need a snapshot.

---

050. Set Operations

THEORY - understand the idea

Set operations compare groups of unique elements. Union (`|`) includes elements from either set; intersection (`&`) includes elements in both; difference (`-`) keeps elements from the left but not the right; symmetric difference (`^`) keeps items found in exactly one set.

CODING EXAMPLE

```
backend = {"Python", "SQL", "HTTP"}
ai = {"Python", "LLMs", "Data"}
print(backend | ai)
print(backend & ai)
print(backend - ai)
print(backend ^ ai)
```

HOW THE CODE WORKS

The expressions answer different questions about the two skill groups: all unique skills, shared skills, only backend skills, and non-shared skills respectively.

PRACTICE EXERCISE

Compare two sets of students enrolled in different courses. Find all students, those taking both, and those taking only the first course.

COMMON PITFALL

Sets are unordered, so the printed order may vary. Compare membership or convert to a sorted list when display order matters.

---

051. List Comprehension

THEORY - understand the idea

A list comprehension creates a new list by transforming each item from an iterable, optionally filtering items with a condition. It is a compact alternative to a simple loop that builds a list. Keep complex logic in a regular loop for readability.

CODING EXAMPLE

```
numbers = [1, 2, 3, 4, 5, 6]
squares = [n * n for n in numbers]
evens = [n for n in numbers if n % 2 == 0]
print(squares)
print(evens)
```

HOW THE CODE WORKS

The first comprehension transforms every item into its square. The second includes only items where the even-number condition passes.

PRACTICE EXERCISE

From a list of prices, create a list with 10% added tax. Then make a second list containing only prices above a chosen threshold.

COMMON PITFALL

A comprehension is not automatically better if several nested conditions make it difficult to read. Prefer clarity over squeezing everything onto one line.

---

052. Dict Comprehension

THEORY - understand the idea

A dictionary comprehension builds a dictionary from an iterable using `{key_expression: value_expression for item in iterable}`. It can transform existing mappings, compute derived values, or filter pairs with an `if` clause.

CODING EXAMPLE

```
numbers = [1, 2, 3, 4]
squares = {n: n * n for n in numbers}
even_squares = {n: n * n for n in numbers if n % 2 == 0}
print(squares)
print(even_squares)
```

HOW THE CODE WORKS

The first dictionary maps each number to its square. The second builds entries only for even numbers.

PRACTICE EXERCISE

Given a list of words, create a dictionary that maps each word to its length. Add a filter to include only words longer than three characters.

COMMON PITFALL

If multiple items produce the same key, a later value overwrites the earlier one. Make sure your key expression matches the intended uniqueness.

---

053. Set Comprehension

THEORY - understand the idea

A set comprehension uses `{expression for item in iterable}` to build a set of unique results. It resembles a list comprehension but removes duplicates and does not promise positional ordering.

CODING EXAMPLE

```
words = ["cat", "dog", "bird", "cat", "dog"]
first_letters = {word[0] for word in words}
lengths = {len(word) for word in words}
print(first_letters)
print(lengths)
```

HOW THE CODE WORKS

The first set contains unique first letters; the second contains unique word lengths. Duplicate results are stored only once.

PRACTICE EXERCISE

From a list of email addresses, build a set of unique domain names after the `@` symbol.

COMMON PITFALL

Use square brackets for a list comprehension. Curly braces with a key-value colon produce a dictionary comprehension; curly braces with one expression produce a set comprehension.

---

054. def

THEORY - understand the idea

`def` defines a function: a named reusable block of code that runs when called. Parameters describe the inputs a function accepts. A well-designed function often performs one clear responsibility and is named with a verb or action.

CODING EXAMPLE

```
def greet(name):
    """Return a friendly greeting."""
    return f"Hello, {name}!"

message = greet("Latif")
print(message)
```

HOW THE CODE WORKS

The function body does not run when `def` is first executed; it runs when `greet("Latif")` is called. The returned string is assigned to `message`.

PRACTICE EXERCISE

Write a function `area_rectangle(width, height)` that returns the area. Call it with at least three pairs of dimensions.

COMMON PITFALL

Function definitions and calls are different actions. Forgetting to call the function means the body will not run.

---

055. return

THEORY - understand the idea

`return` ends the current function call and sends a value back to the caller. If a function reaches the end without returning explicitly, it returns `None`. Returning a value differs from printing it.

CODING EXAMPLE

```
def discount_price(price, percent):
    return price * (1 - percent / 100)

final_price = discount_price(1000, 15)
print(final_price)
```

HOW THE CODE WORKS

The function calculates and returns the discounted value. The caller can store that result, test it, pass it to another function, or print it later.

PRACTICE EXERCISE

Write `is_even(number)` that returns `True` for even numbers and `False` otherwise. Do not print from inside the function.

COMMON PITFALL

Code after `return` in the same execution path will not run. Avoid placing required work after an early return.

---

056. Function Arguments

THEORY - understand the idea

Arguments are the values passed to a function call; parameters are the names listed in the function definition. Positional arguments match by order, while keyword arguments match by parameter name. Using keyword arguments can make calls more readable.

CODING EXAMPLE

```
def describe_product(name, price, in_stock):
    return f"{name}: {price}, stock={in_stock}"

print(describe_product("Pen", 20, True))
print(describe_product(price=20, name="Pen", in_stock=True))
```

HOW THE CODE WORKS

The first call uses parameter order. The second uses keywords, so the order may differ while the values are explicit.

PRACTICE EXERCISE

Define a function that calculates a shipping fee from `weight` and `express`. Call it once positionally and once with keyword arguments.

COMMON PITFALL

Positional arguments generally must come before keyword arguments in a call. Ensure argument names match the function signature.

---

057. Default Arguments

THEORY - understand the idea

A default argument value is used when a caller omits that parameter. Defaults make simple calls convenient while allowing customization. Python evaluates default expressions once when the function is defined, not anew for every call.

CODING EXAMPLE

```
def greet(name, prefix="Hello"):
    return f"{prefix}, {name}!"

print(greet("Sara"))
print(greet("Sara", prefix="Welcome"))
```

HOW THE CODE WORKS

The first call uses `Hello`; the second overrides the default with `Welcome`. Default parameters follow required parameters in the definition.

PRACTICE EXERCISE

Write a `calculate_tax(price, rate=0.05)` function. Call it with the default rate and with a rate of 8%.

COMMON PITFALL

Avoid mutable defaults such as `items=[]` when each call needs a fresh list; the same list would be reused. Use `None` as a sentinel and create a list inside the function.

---

058. *args

THEORY - understand the idea

`*args` collects extra positional arguments into a tuple. It is useful when a function should accept a flexible number of positional inputs. The name `args` is conventional; the asterisk is what matters.

CODING EXAMPLE

```
def total(*numbers):
    return sum(numbers)

print(total(3, 4))
print(total(1, 2, 3, 4, 5))
```

HOW THE CODE WORKS

All positional values after `*numbers` are collected into a tuple. The function can then process that tuple like another iterable.

PRACTICE EXERCISE

Write `average(*scores)` that returns the average of all supplied scores and defines a sensible behavior for zero scores.

COMMON PITFALL

`*args` is not a special reserved name; `*values` would work too. Do not confuse it with the unpacking operator used when calling another function.

---

059. **kwargs

THEORY - understand the idea

`**kwargs` collects extra keyword arguments into a dictionary. It is useful for flexible configuration or forwarding named options. The keys are the keyword names as strings; the values are the supplied objects.

CODING EXAMPLE

```
def show_profile(**details):
    for key, value in details.items():
        print(f"{key}: {value}")

show_profile(name="Latif", role="student", active=True)
```

HOW THE CODE WORKS

The function receives a dictionary with three key-value pairs, then iterates through it. Keyword names become dictionary keys.

PRACTICE EXERCISE

Write a function that accepts any named product details and prints each field. Call it with a product name, price, and category.

COMMON PITFALL

`**kwargs` only collects keyword arguments not otherwise bound by named parameters. Use it thoughtfully; overly flexible APIs can hide what inputs are actually expected.

---

060. lambda

THEORY - understand the idea

A `lambda` creates a small anonymous function containing a single expression. It is handy for short transformations or sort keys, but regular `def` functions are usually clearer for multi-step logic or reusable behavior.

CODING EXAMPLE

```
prices = [40, 5, 120, 25]
prices_sorted = sorted(prices, key=lambda price: price)
words = ["banana", "fig", "apple"]
print(sorted(words, key=lambda word: len(word)))
```

HOW THE CODE WORKS

The first lambda returns its input, so sorting is numeric ascending. The second returns each word length, causing words to be ordered by length.

PRACTICE EXERCISE

Sort a list of dictionaries representing products by their `price` field using a lambda as the key function.

COMMON PITFALL

A lambda is limited to an expression, not a series of statements. Give a complex operation a descriptive name using `def`.

Part 3: OOP, Files & Errors

Model real things, work with files, and make programs handle failures deliberately.

061. Classes

THEORY - understand the idea

A class defines a blueprint for creating objects. It groups data (attributes) and behavior (methods) behind one type. Use a class when a concept has both state and operations that belong together, rather than creating classes for every simple piece of data.

CODING EXAMPLE

```
class Counter:
    def __init__(self, start=0):
        self.value = start

    def increase(self):
        self.value += 1

counter = Counter(3)
counter.increase()
print(counter.value)
```

HOW THE CODE WORKS

`class Counter` defines the type. Creating `Counter(3)` makes an instance with value 3, and its method changes that instance state to 4.

PRACTICE EXERCISE

Create a `BankAccount` class with a starting balance and a `deposit(amount)` method. Add a method to report the current balance.

COMMON PITFALL

A class should have a clear responsibility. Do not introduce class complexity when a simple function or dictionary would be sufficient.

062. Objects

THEORY - understand the idea

An object is an instance of a class. Each instance has identity and can hold its own state, while sharing methods defined by its class. Two objects made from the same class are not automatically the same object.

CODING EXAMPLE

```
class Dog:
    def __init__(self, name):
        self.name = name

first = Dog("Milo")
second = Dog("Luna")
print(first.name, second.name)
print(first is second)
```

HOW THE CODE WORKS

`first` and `second` are separate `Dog` instances with different names. Identity testing returns false because they are distinct objects.

PRACTICE EXERCISE

Create two `Student` objects with different names and semester numbers. Print their data and show that changing one does not change the other.

COMMON PITFALL

Assigning `b = a` does not copy an object; both names refer to the same instance. Learn copying only when you need independent state.

---

063. __init__()

THEORY - understand the idea

`__init__()` is an initializer that runs after Python creates a new instance. It usually sets the instance's starting state. It is not the method that creates the object itself; `__new__()` is involved in object creation.

CODING EXAMPLE

```
class User:
    def __init__(self, username, active=True):
        self.username = username
        self.active = active

user = User("latif")
print(user.username, user.active)
```

HOW THE CODE WORKS

When `User("latif")` is called, the initializer stores the provided username and the default `active` value on that instance.

PRACTICE EXERCISE

Create a `Book` class whose initializer stores title, author, and number of pages. Instantiate two different books.

COMMON PITFALL

Do not explicitly return a value from `__init__()`; it must return `None`. Return values from the initializer are not how you provide the created object.

---

064. self

THEORY - understand the idea

`self` is the conventional name for the current instance passed to an instance method. It lets the method read or change that particular object's attributes. It is an ordinary parameter name, but using `self` is the universal Python convention.

CODING EXAMPLE

```
class Lamp:
    def __init__(self, is_on=False):
        self.is_on = is_on

    def switch(self):
        self.is_on = not self.is_on

lamp = Lamp()
lamp.switch()
print(lamp.is_on)
```

HOW THE CODE WORKS

When `lamp.switch()` is called, Python passes `lamp` as the first argument automatically; inside the method, `self` refers to that instance.

PRACTICE EXERCISE

Add a `rename(new_name)` method to a class that stores a name. Explain which object changes when you call `one.rename("New")`.

COMMON PITFALL

Do not call an instance method on the class without supplying an instance unless you intentionally understand the unbound call form.

---

065. Instance Attributes

THEORY - understand the idea

Instance attributes belong to a particular object and can vary across instances. They are often created in `__init__()` using `self.attribute = value`. Each object can hold its own value for that attribute.

CODING EXAMPLE

```
class Player:
    def __init__(self, name, points=0):
        self.name = name
        self.points = points

one = Player("Ari", 10)
two = Player("Bo", 0)
one.points += 5
print(one.points, two.points)
```

HOW THE CODE WORKS

Updating `one.points` changes only the first player's points. The second object retains its own value of zero.

PRACTICE EXERCISE

Create two product instances, each with a different stock quantity. Update one stock value and show that the other remains unchanged.

COMMON PITFALL

An attribute can be added dynamically in many Python objects, but defining expected attributes clearly in `__init__()` makes classes easier to use and test.

066. Class Attributes

THEORY - understand the idea

A class attribute is stored on the class and is shared as a default across instances unless an instance shadows it with its own attribute. Use class attributes for common settings or constants, not for mutable per-instance state.

CODING EXAMPLE

```
class Employee:
    company = "Example Ltd"

    def __init__(self, name):
        self.name = name

first = Employee("Mina")
second = Employee("Omar")
print(first.company, second.company)
```

HOW THE CODE WORKS

Both objects can read `company` from the class. An assignment like `first.company = "Other"` creates an instance attribute that shadows the class attribute for `first`.

PRACTICE EXERCISE

Create a `Course` class with a shared `institution` attribute and separate `student_name` instance attributes. Experiment with shadowing.

COMMON PITFALL

A mutable class attribute such as `members = []` is shared by all instances. Put per-instance lists in `__init__()` instead.

---

067. Instance Methods

THEORY - understand the idea

An instance method is a function defined in a class that receives the current object as its first parameter, conventionally named `self`. It typically reads or changes instance attributes and is called through an instance.

CODING EXAMPLE

```
class Cart:
    def __init__(self):
        self.items = []

    def add(self, item):
        self.items.append(item)

cart = Cart()
cart.add("Notebook")
print(cart.items)
```

HOW THE CODE WORKS

`add()` receives the cart instance through `self` and modifies its `items` list. The method belongs to the object's behavior.

PRACTICE EXERCISE

Add a `remove_item(name)` instance method to a shopping cart. Decide what should happen if the item does not exist.

COMMON PITFALL

An instance method is different from a class method and a static method. Choose based on whether the operation needs instance state or class-level behavior.

068. @classmethod

THEORY - understand the idea

`@classmethod` makes the first method argument the class itself, conventionally named `cls`. It is often used for alternative constructors or behavior that works with class-level configuration. It also works correctly when subclasses call the method.

CODING EXAMPLE

```
class User:
    def __init__(self, name):
        self.name = name

    @classmethod
    def from_email(cls, email):
        name = email.split("@")[0]
        return cls(name)

user = User.from_email("latif@example.com")
print(user.name)
```

HOW THE CODE WORKS

`from_email()` derives a name and calls `cls(name)`. Using `cls` rather than hard-coding `User` permits a subclass to inherit this constructor and create a subclass instance.

PRACTICE EXERCISE

Add `from_full_name()` as a class method that creates an object from a full name string. Decide how it should handle leading and trailing spaces.

COMMON PITFALL

Do not use `@classmethod` merely because a function sits inside a class. Use it when access to the class is meaningful.

---

069. @staticmethod

THEORY - understand the idea

`@staticmethod` places a function in a class namespace without automatically passing an instance or class. It suits a helper logically related to a class but not dependent on object or class state.

CODING EXAMPLE

```
class PriceRules:
    @staticmethod
    def is_valid(price):
        return price >= 0

print(PriceRules.is_valid(25))
print(PriceRules.is_valid(-1))
```

HOW THE CODE WORKS

No `self` or `cls` is passed automatically. The method checks the given price and returns a Boolean.

PRACTICE EXERCISE

Create a `TextTools` class with a static method that checks whether a string is empty after trimming whitespace.

COMMON PITFALL

If a method needs to use instance data, make it an instance method. If it needs the class itself, consider a class method.

---

070. Inheritance

THEORY - understand the idea

Inheritance lets one class derive behavior and attributes from another. A child class can reuse existing behavior and add or specialize features. It is useful for genuine “is-a” relationships; composition is often preferable when one object simply contains another.

CODING EXAMPLE

```
class Animal:
    def speak(self):
        return "Some sound"

class Cat(Animal):
    def purr(self):
        return "Purr"

cat = Cat()
print(cat.speak(), cat.purr())
```

HOW THE CODE WORKS

`Cat` inherits `speak()` from `Animal` and adds its own `purr()` method. The child can use both unless it changes inherited behavior.

PRACTICE EXERCISE

Create a base `Notification` class with a `send()` method. Create an `EmailNotification` child with additional email-specific data.

COMMON PITFALL

Inheritance can create tight coupling. Avoid large, deep class hierarchies when simpler composition or functions would be easier to maintain.

071. Multiple Inheritance

THEORY - understand the idea

Multiple inheritance allows a class to inherit from more than one base class. Python resolves attribute lookup using the Method Resolution Order (MRO). It can help compose distinct behaviors, but overlapping methods can make programs confusing.

CODING EXAMPLE

```
class Printable:
    def describe(self):
        return "Printable behavior"

class Loggable:
    def log(self):
        return "Logged"

class Report(Printable, Loggable):
    pass

report = Report()
print(report.describe(), report.log())
print(Report.mro())
```

HOW THE CODE WORKS

`Report` inherits methods from both bases. `Report.mro()` shows the order Python searches for attributes and methods.

PRACTICE EXERCISE

Create two small mixin classes that provide distinct methods, then inherit from both. Print the class MRO and explain the lookup order.

COMMON PITFALL

When multiple bases define the same method, understand MRO and cooperative `super()` calls. Avoid complex inheritance webs.

---

072. Encapsulation

THEORY - understand the idea

Encapsulation means keeping data and the operations that manage it together while exposing a controlled interface. Python does not enforce strict private access like some languages: a leading underscore signals internal use, while double underscores trigger name mangling to reduce accidental name conflicts.

CODING EXAMPLE

```
class BankAccount:
    def __init__(self, balance=0):
        self._balance = balance

    def deposit(self, amount):
        if amount <= 0:
            raise ValueError("Deposit must be positive")
        self._balance += amount

    def balance(self):
        return self._balance
```

HOW THE CODE WORKS

The `_balance` name signals that callers should use methods rather than update it casually. `deposit()` validates its input before changing the state.

PRACTICE EXERCISE

Add a withdrawal method that refuses a non-positive amount and refuses to overdraw the account. Write tests for valid and invalid withdrawals.

COMMON PITFALL

A leading underscore is a convention, not a security boundary. Never rely on naming conventions to protect secrets or enforce access control.

073. Polymorphism

THEORY - understand the idea

Polymorphism lets different object types respond to the same operation in their own ways. In Python, this often follows duck typing: code can use an object if it supports the needed behavior, without requiring a shared base class.

CODING EXAMPLE

```
class PDFReport:
    def export(self):
        return "Exporting PDF"

class CSVReport:
    def export(self):
        return "Exporting CSV"

def publish(report):
    print(report.export())

publish(PDFReport())
publish(CSVReport())
```

HOW THE CODE WORKS

`publish()` calls the same `export()` method on different types, and each object provides its own behavior. This makes the caller independent of concrete report classes.

PRACTICE EXERCISE

Create `EmailSender` and `SMSSender` classes with a `send(message)` method. Write a function that calls either implementation through the same interface.

COMMON PITFALL

Polymorphism works best when implementations honor the same behavioral expectations. Similar method names are not enough if return values or side effects differ unexpectedly.

---

074. Abstraction

THEORY - understand the idea

Abstraction exposes the essential interface while hiding implementation details. Python's `abc` module supports abstract base classes, requiring subclasses to implement specified abstract methods before they can be instantiated.

CODING EXAMPLE

```
from abc import ABC, abstractmethod

class PaymentMethod(ABC):
    @abstractmethod
    def pay(self, amount):
        pass

class DemoPayment(PaymentMethod):
    def pay(self, amount):
        return f"Processed {amount} units"

print(DemoPayment().pay(50))
```

HOW THE CODE WORKS

`PaymentMethod` defines the required interface. `DemoPayment` implements `pay()`, so it can be instantiated; the abstract base class itself cannot be instantiated while the method remains abstract.

PRACTICE EXERCISE

Create an abstract `Storage` class with `save(data)`, then implement `MemoryStorage`. Try creating an instance of `Storage` and observe the result.

COMMON PITFALL

Abstraction is not just a class with methods. Use it when callers benefit from a stable contract that can have multiple implementations.

---

075. Method Overriding

THEORY - understand the idea

Method overriding occurs when a child class defines a method with the same name as an inherited method. Calls on child objects use the child implementation, allowing specialization while preserving a shared interface.

CODING EXAMPLE

```
class Animal:
    def speak(self):
        return "Sound"

class Dog(Animal):
    def speak(self):
        return "Woof"

print(Animal().speak())
print(Dog().speak())
```

HOW THE CODE WORKS

The `Dog` implementation overrides `Animal.speak()`. An instance of `Dog` returns `Woof`, while an `Animal` instance returns `Sound`.

PRACTICE EXERCISE

Create a base `Shape.area()` method and override it in `Rectangle` and `Circle`. Test the result for each shape.

COMMON PITFALL

Keep method signatures and expectations compatible when overriding. If a child requires different arguments, callers may not be able to substitute it safely.

076. super()

THEORY - understand the idea

`super()` provides access to methods further along the Method Resolution Order, commonly the parent implementation. It is useful when extending initialization or behavior rather than completely replacing it. In multiple inheritance, cooperative `super()` calls work with the MRO.

CODING EXAMPLE

```
class Person:
    def __init__(self, name):
        self.name = name

class Student(Person):
    def __init__(self, name, semester):
        super().__init__(name)
        self.semester = semester

student = Student("Latif", 4)
print(student.name, student.semester)
```

HOW THE CODE WORKS

`super().__init__(name)` reuses the parent initializer to set `name`; the child then adds `semester`. This avoids repeating initialization logic.

PRACTICE EXERCISE

Create an `Employee` class and a `Manager` subclass. Use `super()` to initialize name and add a team size.

COMMON PITFALL

Calling a parent class by hard-coded name can be less cooperative in multiple inheritance. Prefer `super()` when designing inheritance chains.

077. Magic Methods

THEORY - understand the idea

Magic methods (also called dunder methods) are special methods with double underscores that Python calls for language operations. Examples include `__str__` for readable display, `__repr__` for developer representation, `__len__` for `len()`, and comparison or arithmetic methods.

CODING EXAMPLE

```
class Book:
    def __init__(self, title, pages):
        self.title = title
        self.pages = pages

    def __str__(self):
        return f"{self.title} ({self.pages} pages)"

    def __len__(self):
        return self.pages

book = Book("Python Basics", 180)
print(book)
print(len(book))
```

HOW THE CODE WORKS

`print(book)` calls `__str__()`, while `len(book)` calls `__len__()`. Magic methods let user-defined objects fit naturally into Python syntax.

PRACTICE EXERCISE

Implement `__str__()` for a `Product` object and `__eq__()` to compare products by a chosen identifier. Test both explicitly.

COMMON PITFALL

Only implement magic methods with behavior that matches their expected contract. For example, `__len__()` should return a non-negative integer.

---

078. @property

THEORY - understand the idea

`@property` lets a method be accessed like an attribute. It can compute a value or validate updates while presenting a simple interface to callers. A setter can define what happens when an attribute is assigned.

CODING EXAMPLE

```
class Circle:
    def __init__(self, radius):
        self.radius = radius

    @property
    def diameter(self):
        return self.radius * 2

circle = Circle(5)
print(circle.diameter)
```

HOW THE CODE WORKS

`diameter` is calculated from `radius` each time it is read, so callers use `circle.diameter` rather than `circle.diameter()`.

PRACTICE EXERCISE

Add a `full_name` property that combines a first and last name. Then add validation through a setter to keep a radius positive.

COMMON PITFALL

Properties should usually be fast and unsurprising. Avoid hiding slow network calls or major side effects behind attribute-looking access.

---

079. open()

THEORY - understand the idea

`open()` opens a file and returns a file object used to read or write it. It accepts a path and a mode; text mode is the default. Files should normally be closed with a `with` block so resources are released even if an error occurs.

CODING EXAMPLE

```
from pathlib import Path

path = Path("notes.txt")
with open(path, "w", encoding="utf-8") as file:
    file.write("Learning Python\n")

print(path.exists())
```

HOW THE CODE WORKS

The file opens in write mode and is automatically closed at the end of `with`. `Path.exists()` checks whether the path now exists.

PRACTICE EXERCISE

Create a text file in your project directory and write one short sentence to it. Confirm where the file is created.

COMMON PITFALL

Write mode can replace an existing file's contents. Be deliberate about the path and mode before running write operations.

---

080. read()

THEORY - understand the idea

`file.read()` reads file contents as one string in text mode, or bytes in binary mode. It can read a limited number of characters/bytes with a size argument. Reading the entire contents is convenient for small files but may consume too much memory for very large ones.

CODING EXAMPLE

```
with open("notes.txt", "w", encoding="utf-8") as file:
    file.write("Learning Python\n")

with open("notes.txt", "r", encoding="utf-8") as file:
    contents = file.read()
print(contents)
```

HOW THE CODE WORKS

The first block creates sample text; the second opens the file from the beginning and reads its contents as one string. After reading to the end, another `read()` commonly returns an empty string unless the position is moved.

PRACTICE EXERCISE

Write several lines to a file, read the whole file, and print the result. Then experiment with `read(5)`.

COMMON PITFALL

Opening a missing path in read mode raises `FileNotFoundError`. Ensure the file exists or handle that error.

---

081. write()

THEORY - understand the idea

`file.write(text)` writes text to a text-mode file and returns the number of characters written. It does not automatically add a newline. In write mode, existing content is truncated; append mode writes at the end.

CODING EXAMPLE

```
with open("journal.txt", "w", encoding="utf-8") as file:
    file.write("Day 1: Variables\n")
    file.write("Day 2: Conditions\n")
```

HOW THE CODE WORKS

The
characters create line breaks. Both writes go to the same opened file in order, and the `with` block closes it when complete.

PRACTICE EXERCISE

Write three separate lines to a file, then open it in append mode and add a fourth line without losing the first three.

COMMON PITFALL

Calling `write()` with an integer directly raises a type error in text mode. Convert the value to text or use formatted strings.

---

082. with

THEORY - understand the idea

The `with` statement manages a resource using a context manager. It guarantees the resource's exit logic runs when the block ends, including when an exception occurs. It is widely used for files, locks, database sessions, and other resources.

CODING EXAMPLE

```
with open("hello.txt", "w", encoding="utf-8") as file:
    file.write("The file is managed safely.\n")

# The file is closed here, even if the block had raised an error.
```

HOW THE CODE WORKS

The context manager handles closing the file automatically. That is safer than remembering to call `close()` on every possible path through the program.

PRACTICE EXERCISE

Open a file in a `with` block, deliberately cause an error inside the block after writing, and check that the file object is closed.

COMMON PITFALL

The name after `as` is available inside the block. Avoid using the file after its context has ended.

---

083. File Modes

THEORY - understand the idea

The mode controls how `open()` interacts with a file. Common modes are `"r"` read, `"w"` write/truncate or create, `"a"` append/create, `"x"` exclusive create, `"b"` binary, and `"+"` reading and writing. Choose the mode based on the operation and data type.

CODING EXAMPLE

```
# Replace the contents
with open("sample.txt", "w", encoding="utf-8") as file:
    file.write("first\n")

# Add to the end
with open("sample.txt", "a", encoding="utf-8") as file:
    file.write("second\n")
```

HOW THE CODE WORKS

The first block creates or truncates the file. The second appends to its end. Adding `b` changes text operations to byte operations, and `x` avoids overwriting an existing file.

PRACTICE EXERCISE

Experiment with `w`, `a`, and `x` on a test file. Record which modes create a missing file and which can destroy previous contents.

COMMON PITFALL

Never casually test write modes against an important file. `w` truncates an existing file immediately.

---

084. try

THEORY - understand the idea

`try` marks code that may raise an exception. Python executes the `try` suite and transfers control to a matching `except` suite if a covered exception occurs. Use it where failure is expected and can be handled meaningfully.

CODING EXAMPLE

```
text = "42"
try:
    number = int(text)
    print(100 / number)
except ValueError:
    print("Enter a whole number")
except ZeroDivisionError:
    print("Number must not be zero")
```

HOW THE CODE WORKS

The conversion may raise `ValueError`, and division may raise `ZeroDivisionError`. Separate handlers let the program give a relevant message for each problem.

PRACTICE EXERCISE

Ask for a number and divide 100 by it. Handle non-numeric text and zero separately.

COMMON PITFALL

Do not catch every exception just to hide errors. Catch expected exception types and make failures visible when they cannot be safely recovered from.

---

085. except

THEORY - understand the idea

`except` handles a matching exception raised in the corresponding `try` block. Catch the narrowest useful exception and keep the handler focused on recovery or useful feedback. An exception may also be bound to a name using `as err`.

CODING EXAMPLE

```
try:
    quantity = int("many")
except ValueError as err:
    print("Invalid quantity:", err)
```

HOW THE CODE WORKS

Only a `ValueError` is caught. The exception object `err` contains details that can help diagnose the invalid input.

PRACTICE EXERCISE

Write input parsing that catches `ValueError`, displays a clear message, and lets the user try again in a loop.

COMMON PITFALL

Bare `except:` catches even exceptions such as `KeyboardInterrupt` and `SystemExit`. Avoid it for ordinary application error handling.

---

086. else

THEORY - understand the idea

An `else` block paired with `try` runs only if the `try` suite completes without raising an exception. It helps keep success-path code separate from the lines whose errors you intend to catch.

CODING EXAMPLE

```
try:
    number = int("24")
except ValueError:
    print("Not a whole number")
else:
    print("Conversion succeeded:", number)
```

HOW THE CODE WORKS

The conversion succeeds, so the `else` suite runs. If `int()` raised `ValueError`, the `except` would run and the `else` would be skipped.

PRACTICE EXERCISE

Parse a filename or number in `try`; do the next operation in `else` only after parsing succeeds.

COMMON PITFALL

An exception raised inside the `else` block is not handled by the `except` clauses attached to that same `try`.

---

087. finally

THEORY - understand the idea

`finally` runs when control leaves a `try` statement, whether the operation succeeds or raises an exception. It is intended for cleanup. Context managers are usually an easier and safer way to manage resources like files.

CODING EXAMPLE

```
try:
    print("Work starts")
    result = 10 / 2
except ZeroDivisionError:
    print("Could not divide")
finally:
    print("Cleanup step runs")
```

HOW THE CODE WORKS

The `finally` message runs whether division succeeds or a handled division error occurs. It can also run before an unhandled exception continues propagating.

PRACTICE EXERCISE

Open a resource, simulate both success and failure, and place a cleanup marker in `finally`. Then refactor a file example to use `with`.

COMMON PITFALL

A `return` or exception inside `finally` can override earlier control flow. Keep cleanup code simple and avoid returning from `finally`.

---

088. raise

THEORY - understand the idea

`raise` deliberately signals an exception. Use it when an input or state violates a requirement and the caller should be told that the operation could not proceed. Built-in exceptions such as `ValueError` or `TypeError` are often suitable.

CODING EXAMPLE

```
def set_age(age):
    if age < 0:
        raise ValueError("Age cannot be negative")
    return age

print(set_age(20))
```

HOW THE CODE WORKS

Valid values are returned. A negative age raises `ValueError`; callers can catch it where they can provide useful recovery or feedback.

PRACTICE EXERCISE

Write `withdraw(balance, amount)` that raises `ValueError` for a non-positive amount and raises a suitable error if the amount exceeds the balance.

COMMON PITFALL

Do not return a string like `"ERROR"` for every failure when callers need to distinguish errors. Exceptions let failure be handled explicitly.

---

089. Custom Exceptions

THEORY - understand the idea

A custom exception is a class derived from `Exception` that represents a domain-specific failure. It makes error handling clearer when a program needs to distinguish, for example, an unavailable product from invalid input.

CODING EXAMPLE

```
class OutOfStockError(Exception):
    """Raised when inventory cannot satisfy an order."""


def reserve(stock, quantity):
    if quantity > stock:
        raise OutOfStockError("Not enough stock")
    return stock - quantity

print(reserve(10, 3))
```

HOW THE CODE WORKS

`OutOfStockError` names a meaningful business failure. Code can catch that specific exception without accidentally swallowing unrelated errors.

PRACTICE EXERCISE

Create `InsufficientFundsError` and raise it when a withdrawal is larger than the account balance. Catch it at the calling layer and display a friendly message.

COMMON PITFALL

Custom exception types should generally inherit from `Exception`, not `BaseException`. Keep the exception hierarchy small and purposeful.

---

090. assert

THEORY - understand the idea

`assert condition, message` checks an assumption and raises `AssertionError` if it is false. It is useful for developer-facing invariants and tests, not for validating untrusted user input, because Python can disable assertions with optimization flags.

CODING EXAMPLE

```
quantity = 3
assert quantity > 0, "Quantity should be positive"
print("Proceed with order")
```

HOW THE CODE WORKS

When the condition is true, execution continues. If `quantity` were zero or negative, an `AssertionError` would be raised when assertions are enabled.

PRACTICE EXERCISE

Add assertions around internal assumptions in a small calculation function. Then replace user-input validation with explicit `if`/`raise` logic.

COMMON PITFALL

Do not rely on `assert` to enforce security or business rules in production. Use explicit validation and exceptions for conditions that must always be checked.

Part 4: Advanced Python & Ecosystem

Understand modules, iteration, functional tools, concurrency, and useful standard-library patterns.

091. import

THEORY - understand the idea

`import` loads a module and binds its module name in the current namespace. A module is a Python file or importable module-like component containing reusable code. Importing avoids copying and pasting functions across files.

CODING EXAMPLE

```
import math
import random

print(math.sqrt(81))
print(random.choice(["red", "blue", "green"]))
```

HOW THE CODE WORKS

`math` and `random` are imported as module objects; functions are accessed with dot notation. `random.choice()` may return a different item on each run.

PRACTICE EXERCISE

Import `datetime` and print today's date. Then import `statistics` and calculate the mean of a list.

COMMON PITFALL

Imports can execute top-level code in a module. Keep scripts safe to import by placing run-only behavior inside `if __name__ == "__main__":`.

---

092. from ... import

THEORY - understand the idea

`from module import name` imports selected names directly into the current namespace. It can reduce repetition, but it may hide where a name came from and can create collisions if two modules provide the same name.

CODING EXAMPLE

```
from math import sqrt, pi

print(sqrt(49))
print(pi)
```

HOW THE CODE WORKS

After this import, `sqrt` and `pi` can be used without prefixing `math.`. The original module name is not required for those names.

PRACTICE EXERCISE

Import `date` from `datetime` and display a date. Then compare readability with `import datetime`.

COMMON PITFALL

Avoid `from module import *`; it can clutter the namespace and make it difficult to track where names are defined.

---

093. Modules

THEORY - understand the idea

A module is a `.py` file that can define variables, functions, classes, and executable statements. Splitting a project into modules helps separate responsibilities and reuse code. A module should have a clear purpose and a sensible filename.

CODING EXAMPLE

```
import statistics

scores = [70, 80, 90]
average = statistics.mean(scores)
print(average)
```

HOW THE CODE WORKS

`statistics` is a module from the standard library. Its `mean()` function is available through the module name, illustrating how modules group related functionality. A project can create its own `.py` modules in the same way.

PRACTICE EXERCISE

Create `math_tools.py` with two small functions, then import and call them from `main.py` in the same directory.

COMMON PITFALL

Choose filenames that do not shadow important standard-library modules such as `json.py`, `random.py`, or `typing.py`.

---

094. Packages

THEORY - understand the idea

A package groups related modules under a shared namespace. A typical regular package is a directory containing an `__init__.py` file, though namespace packages can work without one. Packages help larger projects organize code by domain or responsibility.

CODING EXAMPLE

```
from email.utils import parseaddr

name, address = parseaddr("Latif <latif@example.com>")
print(name)
print(address)
```

HOW THE CODE WORKS

`email` is a standard-library package and `utils` is a module inside it. The dotted import path selects a function from that module. Your own projects can use the same package-and-module pattern.

PRACTICE EXERCISE

Create a package named `utilities` with `text.py` and `numbers.py`. Import one function from each module into a small runner script.

COMMON PITFALL

Run modules from a predictable project root and use clear package names. Import errors often result from unexpected working directories or name collisions.

---

095. pip

THEORY - understand the idea

`pip` is Python's package installer, used to install third-party libraries from package indexes or other sources. Install packages into the environment used by your project, and record dependencies so the project can be recreated.

TERMINAL COMMANDS - run in your shell

```
# Run these commands in a terminal, not inside a Python file:
python -m pip install requests
python -m pip show requests
python -m pip freeze > requirements.txt
```

HOW THE CODE WORKS

`python -m pip` runs pip with the selected Python interpreter, reducing ambiguity about which installation receives the package. `pip freeze` writes installed package versions to a requirements file.

PRACTICE EXERCISE

Install a small third-party library in a virtual environment, import it in Python, and create a `requirements.txt` file.

COMMON PITFALL

Installing packages globally can create conflicts between projects. Prefer a dedicated virtual environment and do not blindly install unknown packages.

---

096. Virtual Environments

THEORY - understand the idea

A virtual environment is an isolated Python environment for one project's packages. It helps prevent dependency versions from one project affecting another. Create it for each project and use the environment's Python and pip commands.

TERMINAL COMMANDS - run in your shell

```
# In a terminal, from the project folder:
python -m venv .venv

# Activate on Linux/macOS:
source .venv/bin/activate

# Then install project requirements:
python -m pip install -r requirements.txt
```

HOW THE CODE WORKS

`venv` creates the `.venv` directory. Activation adjusts the shell so `python` and `pip` usually point to that environment. On Windows, activation uses `.venv\Scripts\activate` in PowerShell or Command Prompt.

PRACTICE EXERCISE

Create a virtual environment for a fresh folder, install one library inside it, and verify the interpreter path using `python -c "import sys; print(sys.executable)"`.

COMMON PITFALL

Do not commit the entire `.venv` folder to Git. Commit dependency declarations such as `requirements.txt` and document how to recreate the environment.

---

097. __name__

THEORY - understand the idea

`__name__` is a module-level variable. When a file is run directly, its value is normally `"__main__"`; when imported, it is typically the module's import name. This supports files that work both as reusable modules and runnable scripts.

CODING EXAMPLE

```
def main():
    print("Program entry point")

if __name__ == "__main__":
    main()
```

HOW THE CODE WORKS

Importing this file defines `main()` but does not call it. Running the file directly satisfies the condition and calls `main()`.

PRACTICE EXERCISE

Create a module with one reusable function and a `main()` function. Import it from another file and confirm the demonstration output only appears when the file is run directly.

COMMON PITFALL

Keep reusable definitions at module level and put script-only actions under the guard. This reduces surprising behavior during imports and testing.

---

098. __iter__()

THEORY - understand the idea

`__iter__()` is a special method used to obtain an iterator from an iterable. A custom iterable often implements this method to return an iterator over its data. Many built-in objects such as lists already do this.

CODING EXAMPLE

```
class Countdown:
    def __init__(self, start):
        self.start = start

    def __iter__(self):
        return iter(range(self.start, 0, -1))

for number in Countdown(3):
    print(number)
```

HOW THE CODE WORKS

`Countdown` returns an iterator over the generated range. The `for` loop calls the iteration protocol automatically and receives 3, 2, then 1.

PRACTICE EXERCISE

Create a `Playlist` iterable that returns its songs in order. Use it in a `for` loop and convert it to a list.

COMMON PITFALL

An iterable and an iterator are related but different: an iterable can provide an iterator; an iterator also remembers its current position and implements `__next__()`.

---

099. iter()

THEORY - understand the idea

`iter(object)` asks an iterable for an iterator. An iterator remembers where it is and supplies one item at a time. The normal `for` loop uses `iter()` behind the scenes, so you rarely need to call it directly unless learning or implementing iteration.

CODING EXAMPLE

```
colors = ["red", "green", "blue"]
iterator = iter(colors)
print(next(iterator))
print(next(iterator))
```

HOW THE CODE WORKS

`iter(colors)` creates an iterator over the list. Each `next()` call advances it and returns the next item without changing the underlying list.

PRACTICE EXERCISE

Create an iterator from a list of four tasks and manually retrieve the first two tasks with `next()`.

COMMON PITFALL

An iterator is usually exhausted after all its elements are consumed; asking for another item raises `StopIteration`.

---

100. next()

THEORY - understand the idea

`next(iterator)` asks an iterator for its next item. It raises `StopIteration` when no items remain, unless a default value is supplied. For loops handle this stopping signal automatically.

CODING EXAMPLE

```
numbers = iter([10, 20])
print(next(numbers))
print(next(numbers))
print(next(numbers, "Finished"))
```

HOW THE CODE WORKS

The first two calls return 10 and 20. The third call returns `Finished` because the iterator has no more items.

PRACTICE EXERCISE

Use `iter()` and `next()` to consume values from a list one at a time. Compare the result with a regular loop.

COMMON PITFALL

Calling `next()` repeatedly without handling exhaustion can raise `StopIteration`. Use a `for` loop unless manual control is useful.

---

101. Generators

THEORY - understand the idea

A generator is a convenient way to produce a sequence lazily, one item at a time, instead of storing all results in memory. Generator functions use `yield`; generator expressions use parentheses. Generators are especially useful for large or streamed data.

CODING EXAMPLE

```
def countdown(start):
    while start > 0:
        yield start
        start -= 1

for value in countdown(3):
    print(value)
```

HOW THE CODE WORKS

Calling `countdown(3)` creates a generator without immediately running the whole body. Each iteration resumes it until the next `yield` or until it finishes.

PRACTICE EXERCISE

Write a generator that yields the first `n` even numbers. Consume it in a loop and convert it to a list for comparison.

COMMON PITFALL

Generators are usually consumed once. If you need to iterate over values multiple times, create a new generator or store the values.

---

102. yield

THEORY - understand the idea

`yield` pauses a generator function and produces a value to its caller. When iteration continues, execution resumes immediately after the `yield`, preserving local variables and control state. This is different from `return`, which ends the function call.

CODING EXAMPLE

```
def steps():
    print("start")
    yield 1
    print("middle")
    yield 2
    print("end")

for step in steps():
    print("got", step)
```

HOW THE CODE WORKS

The output interleaves generator-side messages with the loop because each `yield` pauses execution. The `end` message appears only when the generator is resumed after its final yielded value.

PRACTICE EXERCISE

Write a generator that yields the lines of a short checklist one by one. Add a print before and after each yield to observe when execution resumes.

COMMON PITFALL

Using `yield` changes a function into a generator function. Its body does not run in full at the moment it is called.

---

103. Decorators

THEORY - understand the idea

A decorator receives a function or class and returns a replacement or wrapped version. The `@decorator` syntax applies it at definition time. Decorators are commonly used for logging, timing, authorization, caching, and framework registration.

CODING EXAMPLE

```
from functools import wraps

def announce(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        print("Calling", func.__name__)
        return func(*args, **kwargs)
    return wrapper

@announce
def greet(name):
    return f"Hello, {name}"

print(greet("Latif"))
```

HOW THE CODE WORKS

`announce()` wraps the original function. When `greet()` is called, the wrapper prints a message and forwards the arguments, then returns the original result. `wraps` preserves useful function metadata.

PRACTICE EXERCISE

Create a decorator that prints `starting` and `finished` around a function call, including when the function returns a value.

COMMON PITFALL

Decorator wrappers should usually forward `*args` and `**kwargs` and return the wrapped result. Real production decorators should also consider exceptions and metadata.

104. Closures

THEORY - understand the idea

A closure is an inner function that remembers variables from the enclosing function even after the outer function has returned. Closures can keep small pieces of state or configure behavior without creating a class.

CODING EXAMPLE

```
def make_multiplier(factor):
    def multiply(number):
        return number * factor
    return multiply

double = make_multiplier(2)
triple = make_multiplier(3)
print(double(5), triple(5))
```

HOW THE CODE WORKS

Each returned function remembers its own `factor`. `double` keeps 2 and `triple` keeps 3, even after the calls to `make_multiplier()` have completed.

PRACTICE EXERCISE

Write `make_prefix(prefix)` that returns a function which adds that prefix to any text. Create two differently configured functions.

COMMON PITFALL

When modifying a captured variable, `nonlocal` may be required. Be mindful of shared mutable state when closures are used in asynchronous or concurrent code.

---

105. Recursion

THEORY - understand the idea

Recursion is when a function calls itself to solve a smaller version of a problem. A correct recursive design needs a base case that stops and a progress step that moves toward the base case. Recursion can be elegant for trees and divide-and-conquer problems.

CODING EXAMPLE

```
def factorial(n):
    if n < 0:
        raise ValueError("n must be non-negative")
    if n <= 1:
        return 1
    return n * factorial(n - 1)

print(factorial(5))
```

HOW THE CODE WORKS

`factorial(0)` and `factorial(1)` stop at the base case. Larger values multiply `n` by the result for `n - 1`. For 5, the result is 120.

PRACTICE EXERCISE

Write a recursive function to sum integers from 1 through `n`. Test the base case and a few larger values; define behavior for negative input.

COMMON PITFALL

Without a base case or progress toward it, recursion may continue until Python raises `RecursionError`. An iterative solution is often better for simple loops.

---

106. Context Managers

THEORY - understand the idea

A context manager defines setup and cleanup behavior for a `with` statement. Objects can implement `__enter__()` and `__exit__()`, or use `contextlib`. It helps manage resources and ensure cleanup follows a predictable path.

CODING EXAMPLE

```
from contextlib import contextmanager

@contextmanager
def managed_message():
    print("enter")
    try:
        yield "resource"
    finally:
        print("exit")

with managed_message() as value:
    print(value)
```

HOW THE CODE WORKS

The code before `yield` is setup, the yielded value becomes `value`, and the `finally` block runs as the context exits, including if the block raises an exception.

PRACTICE EXERCISE

Create a context manager that prints when a timed operation begins and ends. Use `try/finally` so the ending message appears even if the body fails.

COMMON PITFALL

For actual files, use built-in `open()` with `with`. Write a custom context manager only when it encapsulates a meaningful reusable setup/cleanup pattern.

---

107. map()

THEORY - understand the idea

`map(function, iterable)` lazily applies a function to every item from an iterable. It returns an iterator, not a list. A comprehension is often more readable in simple cases, while `map()` can be useful when a named function already exists.

CODING EXAMPLE

```
numbers = ["1", "2", "3"]
converted = map(int, numbers)
print(list(converted))
```

HOW THE CODE WORKS

`int` is applied to each string when the iterator is consumed. `list()` forces all results to be computed and stored in a list.

PRACTICE EXERCISE

Use `map()` to convert a list of numeric strings to integers. Then rewrite the same operation with a list comprehension and compare clarity.

COMMON PITFALL

Map objects are single-pass iterators. Once consumed, they do not automatically restart.

---

108. filter()

THEORY - understand the idea

`filter(function, iterable)` yields only items for which the function returns a truthy result. If the function is `None`, it filters by truthiness. Like `map()`, it is lazy and returns an iterator.

CODING EXAMPLE

```
numbers = [1, 2, 3, 4, 5, 6]
evens = filter(lambda n: n % 2 == 0, numbers)
print(list(evens))
```

HOW THE CODE WORKS

The predicate returns true for even numbers. `filter()` passes through those original values and skips the others.

PRACTICE EXERCISE

Filter a list of usernames to keep only names longer than five characters. Write a second version using a comprehension.

COMMON PITFALL

For basic filtering, `[x for x in items if condition]` is often easier to read than `filter(lambda ...)`.

---

109. any()

THEORY - understand the idea

`any(iterable)` returns `True` if at least one item is truthy, and `False` when every item is falsey or the iterable is empty. It stops as soon as it finds a truthy item, so it can be efficient.

CODING EXAMPLE

```
scores = [42, 38, 76, 49]
passed = any(score >= 50 for score in scores)
print(passed)
print(any([]))
```

HOW THE CODE WORKS

The generator expression checks scores until it finds one at least 50; the result is true. The empty iterable contains no truthy item, so `any([])` is false.

PRACTICE EXERCISE

Given a list of product stock quantities, check whether at least one product is out of stock.

COMMON PITFALL

`any()` checks truthiness, not a hidden business rule. Supply an explicit condition when a specific meaning of “true” is required.

---

110. all()

THEORY - understand the idea

`all(iterable)` returns `True` when every item is truthy, and also returns `True` for an empty iterable. That empty-input behavior is called vacuous truth and can surprise beginners. `all()` stops at the first falsey item.

CODING EXAMPLE

```
scores = [70, 82, 91]
all_passed = all(score >= 50 for score in scores)
print(all_passed)
print(all([]))
```

HOW THE CODE WORKS

All scores pass the condition, so the result is true. `all([])` is also true because there is no item that violates the condition.

PRACTICE EXERCISE

Check whether all usernames in a list are non-empty after trimming. Decide whether an empty list should count as valid in your application.

COMMON PITFALL

If empty input should be rejected, check `bool(items)` separately rather than assuming `all()` returns false.

---

111. async

THEORY - understand the idea

`async def` defines an asynchronous function, also called a coroutine function. Calling it creates a coroutine object; it does not run the function to completion immediately. Async functions are useful when work can wait for I/O without blocking other coroutines.

CODING EXAMPLE

```
import asyncio

async def greet():
    print("Hello")
    await asyncio.sleep(0.1)
    print("Goodbye")

asyncio.run(greet())
```

HOW THE CODE WORKS

`asyncio.run()` starts an event loop and runs the coroutine. `await` lets the coroutine pause while the sleep operation completes, giving the event loop a chance to run other work.

PRACTICE EXERCISE

Write two asynchronous tasks that each wait briefly and print start/end messages. Run them concurrently with `asyncio.gather()`.

COMMON PITFALL

An `async def` function is not automatically concurrent just because it is declared async. It must be awaited or scheduled on an event loop.

---

112. await

THEORY - understand the idea

`await` pauses a coroutine until an awaitable operation completes, while allowing the event loop to schedule other tasks. It can be used inside `async def` functions. Awaiting a blocking synchronous operation does not magically make that operation non-blocking.

CODING EXAMPLE

```
import asyncio

async def fetch_demo():
    await asyncio.sleep(0.2)
    return {"status": "ok"}

async def main():
    result = await fetch_demo()
    print(result)

asyncio.run(main())
```

HOW THE CODE WORKS

`await fetch_demo()` suspends `main()` until `fetch_demo()` returns its dictionary. The event loop can perform other work during an asynchronous wait.

PRACTICE EXERCISE

Create a coroutine that waits using `asyncio.sleep()` and returns a message. Await it from another coroutine.

COMMON PITFALL

You cannot use `await` at arbitrary places in normal synchronous functions. Also, avoid blocking calls such as `time.sleep()` inside async code.

113. Coroutines

THEORY - understand the idea

A coroutine is a computation that can suspend and resume, often while waiting for I/O. Python coroutines are commonly defined with `async def` and use `await`. Coroutines allow structured concurrency, but they do not automatically run on separate CPU cores.

CODING EXAMPLE

```
import asyncio

async def task(name, delay):
    await asyncio.sleep(delay)
    return f"{name} complete"

async def main():
    results = await asyncio.gather(
        task("A", 0.2), task("B", 0.1)
    )
    print(results)

asyncio.run(main())
```

HOW THE CODE WORKS

`gather()` schedules both coroutine operations so their waiting periods overlap. Results are returned in the order the awaitables were provided, not necessarily completion order.

PRACTICE EXERCISE

Create three coroutines with different delays. Use `gather()` to run them and report the returned results.

COMMON PITFALL

Use async primarily for workloads with awaitable I/O. Heavy CPU-bound work usually needs a different strategy.

---

114. asyncio

THEORY - understand the idea

`asyncio` is Python's standard library for writing asynchronous code with an event loop, tasks, and awaitables. It is particularly useful for many simultaneous network operations when the libraries involved support async I/O.

CODING EXAMPLE

```
import asyncio

async def worker(number):
    await asyncio.sleep(0.1)
    return number * 10

async def main():
    tasks = [worker(n) for n in range(4)]
    results = await asyncio.gather(*tasks)
    print(results)

asyncio.run(main())
```

HOW THE CODE WORKS

`worker()` creates coroutine objects; `gather()` awaits all of them together, and `asyncio.run()` manages the top-level event loop for this script.

PRACTICE EXERCISE

Build an async demo that runs five simulated API requests with `asyncio.sleep()`. Measure elapsed time and compare sequential waiting with `gather()`.

COMMON PITFALL

In notebooks or environments that already run an event loop, calling `asyncio.run()` may be invalid. Use that environment's supported top-level await pattern instead.

---

115. Threading

THEORY - understand the idea

`threading` lets a process run multiple threads that share memory. Threads are useful for many blocking I/O operations, such as waiting on files or network calls. In standard CPython builds, the Global Interpreter Lock usually limits parallel execution of Python bytecode for CPU-bound work.

CODING EXAMPLE

```
import threading
import time

def download(name):
    print("Starting", name)
    time.sleep(0.2)  # Simulates waiting for I/O
    print("Finished", name)

threads = [threading.Thread(target=download, args=(name,))
           for name in ("A", "B")]
for thread in threads:
    thread.start()
for thread in threads:
    thread.join()
```

HOW THE CODE WORKS

Each thread begins running the function independently. `join()` waits for each thread to finish before the main program continues. The sleep simulates an I/O wait, not useful CPU work.

PRACTICE EXERCISE

Run three simulated file or network waits using threads. Compare with a sequential version and measure the elapsed time.

COMMON PITFALL

Shared mutable state requires care: protect shared updates with appropriate synchronization or avoid sharing the state.

---

116. Multiprocessing

THEORY - understand the idea

`multiprocessing` runs work in separate processes, each with its own Python interpreter and memory space. It can use multiple CPU cores for CPU-bound work, but processes have startup and data-transfer overhead.

CODING EXAMPLE

```
from concurrent.futures import ProcessPoolExecutor

def square(number):
    return number * number

if __name__ == "__main__":
    with ProcessPoolExecutor() as pool:
        print(list(pool.map(square, [1, 2, 3, 4])))
```

HOW THE CODE WORKS

The worker function can be executed in child processes. The main guard is important, especially on platforms that start workers by importing the module.

PRACTICE EXERCISE

Use a process pool to calculate squares of a larger list of numbers, then compare it to a simple list comprehension. Discuss when the process overhead is worth it.

COMMON PITFALL

Process worker functions and their inputs/results generally need to be pickleable. Keep process-starting code under the main guard.

---

117. dataclass

THEORY - understand the idea

`dataclasses.dataclass` generates common methods for classes that primarily store data, such as an initializer and a readable representation. It reduces boilerplate while allowing ordinary methods, validation, and type annotations.

CODING EXAMPLE

```
from dataclasses import dataclass

@dataclass
class Product:
    name: str
    price: float
    in_stock: bool = True

product = Product("Mouse", 2500.0)
print(product)
```

HOW THE CODE WORKS

The decorator generates `__init__()` and `__repr__()` by default. The `in_stock` field uses a default value, so callers may omit it.

PRACTICE EXERCISE

Create a `Student` dataclass with name, semester, and an optional GPA. Instantiate two students and compare their representations.

COMMON PITFALL

Use `field(default_factory=list)` for a mutable default such as a list. Do not share one mutable list between instances accidentally.

---

118. Type Hints

THEORY - understand the idea

Type hints annotate expected kinds of values for readers, editors, and static analysis tools. Python usually does not enforce these hints at runtime. They improve maintainability when used consistently and kept aligned with the implementation.

CODING EXAMPLE

```
def total_price(price: float, quantity: int) -> float:
    return price * quantity

result: float = total_price(12.5, 3)
print(result)
```

HOW THE CODE WORKS

Parameters and return value are annotated. The interpreter still allows many mismatches at runtime; tools such as type checkers can detect some problems before execution.

PRACTICE EXERCISE

Add type hints to a function that accepts a list of prices and returns their total. Choose suitable annotations for empty and optional inputs.

COMMON PITFALL

Type hints are not input validation or security controls. Validate external data at runtime, for example with explicit checks or a validation library.

---

119. Regular Expressions

THEORY - understand the idea

`re` provides regular-expression pattern matching for text. It is useful for locating patterns, extracting groups, replacing text, and splitting strings. Patterns can be powerful, but for complex input formats, a dedicated parser may be safer.

CODING EXAMPLE

```
import re

text = "Contact: latif@example.com"
match = re.search(r"[\w.+-]+@[\w.-]+\.[A-Za-z]{2,}", text)
if match:
    print(match.group())
```

HOW THE CODE WORKS

`re.search()` finds the first substring matching the pattern. A raw string (`r"..."`) helps write backslashes in patterns without extra Python-level escaping.

PRACTICE EXERCISE

Extract all digit sequences from a string using `re.findall()`. Then validate a few sample inputs and identify cases your pattern does not handle.

COMMON PITFALL

Regular expressions for email, phone, and URL validation can be imperfect. Do not assume a simple pattern covers every valid real-world case.

---

120. JSON Handling

THEORY - understand the idea

`json` converts between JSON text and Python values. `json.loads()` parses a JSON string; `json.dumps()` serializes a Python value to a string. For files, use `json.load()` and `json.dump()`. JSON supports objects, arrays, strings, numbers, booleans, and null, with constraints compared with Python.

CODING EXAMPLE

```
import json

data = {"name": "Latif", "skills": ["Python", "FastAPI"], "active": True}
text = json.dumps(data, indent=2)
print(text)
restored = json.loads(text)
print(restored["skills"][0])
```

HOW THE CODE WORKS

`dumps()` produces JSON text, including formatted indentation. `loads()` parses that text back into a Python dictionary and list structure.

PRACTICE EXERCISE

Create a product dictionary, serialize it to a JSON string, parse it back, and verify its price and stock fields. Then write/read the object from a JSON file.

COMMON PITFALL

JSON text uses `true`, `false`, and `null`, whereas Python uses `True`, `False`, and `None`. Do not use `eval()` to parse JSON; use the `json` module.

Final Review: Turn Reading into Skill

Finishing the pages is not the same as mastering the concepts. Use this review checklist to verify that you can work independently.

- ☐ I can use variables, types, conversion, conditions, loops, and operators without copying an example.
- ☐ I can choose between lists, tuples, sets, and dictionaries based on how the data will be used.
- ☐ I can write small functions with clear parameters and return values, and I understand common argument patterns.
- ☐ I can explain instance attributes, class attributes, methods, inheritance, and when a simpler function is better.
- ☐ I can read and write files safely with `with`, select a file mode, and handle expected errors.
- ☐ I can organize code into modules and packages and use a virtual environment for a project.
- ☐ I can explain iterator/generator behavior and recognize where lazy processing helps.
- ☐ I can write a simple decorator and explain a closure without memorizing the code.
- ☐ I can distinguish asynchronous I/O from threads and processes at a high level.
- ☐ I can use data classes, type hints, regular expressions, and JSON appropriately for small tasks.

Suggested Mini-Projects

1. Command-line expense tracker

Use input, loops, lists/dictionaries, functions, validation, JSON storage, and exception handling.

2. Inventory manager

Model products with classes or dataclasses; add, remove, search, sort, and save stock values to a file.

3. Log analyzer

Read a text file, use string methods and regular expressions, count events with dictionaries or Counter, and export a JSON summary.

4. Async request simulator

Use coroutines, `asyncio.gather()`, and timing to compare concurrent waits with sequential waits.

A Note on “Advanced” Python

Advanced does not mean using the most complicated feature. Good Python engineering means choosing the simplest feature that clearly solves the problem, testing edge cases, handling failures, and making code easy for another developer to understand.